

Temperatura-y-humedad

Monitoreo de temperatura y humedad con Arduino UNO R4 WiFi, sensor DHT22, Make.com y Telegram

Repositorio	github.com/Ruben-N1/Temperatura-y-humedad
Autor	Ruben-N1
Fecha del reporte	27 de septiembre de 2026
Rama analizada	main

Este reporte resume la estructura, el código, el estado actual y observaciones técnicas del repositorio público **Temperatura-y-humedad**, un proyecto escolar de Arduino que lee temperatura y humedad con un sensor DHT22 y envía alertas a través de Make.com y un bot de Telegram.

1. Descripción general

El proyecto usa un módulo **DHT22** para detectar si la temperatura sobrepasa cierto umbral. Cuando esto ocurre, el Arduino envía la información a **Make.com** mediante un webhook; desde ahí, un escenario con un módulo de bot de Telegram envía un mensaje al usuario informándole de la temperatura (según la descripción del propio README del repositorio).

Objetivo de aprendizaje declarado: aprender a hacer uso correcto de Arduino, LEDs, resistencias y el módulo DHT22.

Material utilizado

- Protoboard
- Arduino UNO R4 WiFi
- 4 jumpers
- Resistencia
- LED
- Sensor DHT22

2. Estructura del repositorio

El repositorio está organizado en cuatro carpetas de propósito específico más el README principal:

Carpeta / archivo	Contenido actual
-------------------	------------------

codigo/	Temperatura.ino — sketch principal del Arduino
imagenes/	Fotografía del circuito armado (image_2026-09-26_223029875.png)
resultados/	Sin archivos visibles / pendiente de llenar
video/	Sin archivos visibles / pendiente de llenar
README.md	Documentación con secciones base, varias aún vacías

El código fuente (codigo/Temperatura.ino) usa la librería WiFiS3.h, propia de las placas Arduino UNO R4 WiFi.

3. Diagrama del circuito

El README incluye una fotografía del montaje físico en lugar de un diagrama esquemático. Se observa una protoboard con un LED verde encendido y su resistencia asociada, conectados mediante 4 jumpers (amarillo, verde, gris y rojo) a un Arduino UNO R4 WiFi, el cual está alimentado por cable USB.

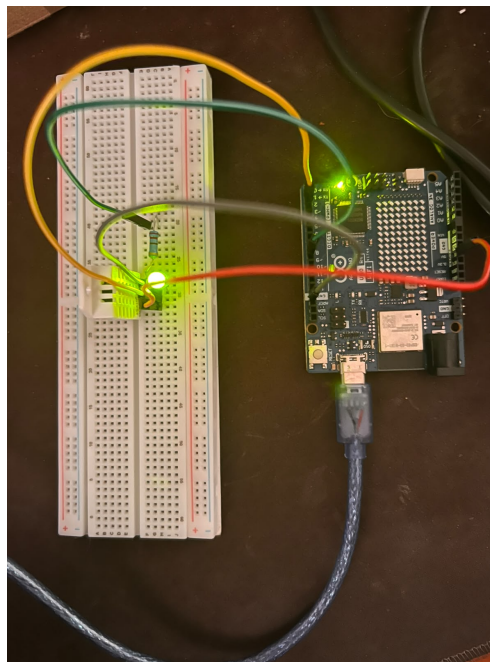


Figura 1. Montaje físico del circuito (imagenes/image_2026-09-26_223029875.png)

4. Análisis del código — codigo/Temperatura.ino

4.1 Configuración inicial

El sketch define el pin del sensor (DHTPIN 2, tipo DHT22) y un LED indicador en el pin 8. En `setup()` se inicializa el puerto serie, se configura el LED, se arranca el sensor DHT y se conecta a la red WiFi definida por SSID y contraseña, esperando a que la placa reciba una IP por DHCP antes de continuar.

4.2 Ciclo principal (loop)

- Lee temperatura y humedad del DHT22; si la lectura falla (NaN), lo reporta por serie y reintenta en 2 s.
- Imprime los valores leídos por el puerto serie.
- Si la temperatura supera **26.0 °C**, enciende el LED del pin 8 y, solo la primera vez que se cruza el umbral (bandera `avisoEnviado`), llama a `enviarMake()` para notificar.
- Si la temperatura vuelve a 26 °C o menos, apaga el LED y reinicia la bandera para permitir un nuevo aviso en el siguiente pico.
- Repite el ciclo cada 2 segundos.

4.3 Envío de datos a Make.com

La función `enviarMake()` abre una conexión TLS (`WiFiSSLClient`) al host del webhook de Make, arma un JSON manual con los campos temperatura y humedad, y hace un POST HTTP/1.1 con las cabeceras correspondientes. Luego espera hasta 5 segundos una respuesta del servidor y cierra la conexión.

4.4 Fragmento representativo

```
if (temperatura > 26.0) {  
    digitalWrite(LEDPIN, HIGH);  
    if (!avisoEnviado) {  
        enviarMake(temperatura, humedad);  
        avisoEnviado = true;  
    }  
} else {  
    digitalWrite(LEDPIN, LOW);  
    avisoEnviado = false;  
}
```

5. Observaciones técnicas y recomendaciones

Buenas prácticas ya presentes

- Uso correcto de bandera de estado (`avisoEnviado`) para evitar enviar notificaciones repetidas mientras la temperatura se mantiene alta.
- Manejo del caso de lectura fallida del sensor (`isnan`) antes de usar los datos.
- Espera activa razonable a la conexión WiFi y a la asignación de IP antes de operar.

Puntos a mejorar

- **Credenciales en texto plano:** el SSID, la contraseña WiFi y la ruta del webhook de Make están escritos directamente en el código fuente, que es público. Se recomienda moverlos a un archivo de configuración no versionado (por ejemplo `secrets.h` agregado a `.gitignore`) y rotar la contraseña WiFi actual.

- El umbral de temperatura (26.0 °C) está fijo en el código; podría exponerse como constante documentada o incluso recibirse desde Make para no requerir recompilar el sketch al ajustarlo.
 - No hay verificación del código de estado HTTP de la respuesta de Make, solo se imprime el texto crudo recibido; sería útil validar por ejemplo que la respuesta indique éxito (200).
 - El README define secciones para Código, Video, Evidencias, Reporte, Gráficas, Tablas de datos, Observaciones, Conclusiones y Resultados, pero todas aparecen vacías por ahora.
-

6. Estado de la documentación (README.md)

Secciones completas: Nombre del proyecto, Descripción, Objetivos de aprendizaje, Material utilizado y Diagrama del circuito.

Secciones creadas pero aún sin contenido: Código, Video del funcionamiento, Evidencias de armado, Reporte, Gráficas, Tablas de datos, Observaciones sobre el comportamiento del sistema, Conclusiones y Resultados. Las carpetas resultados/ y video/ existen en el repositorio pero, al momento de este reporte, no muestran archivos.

7. Conclusión

El proyecto tiene una base funcional sólida: el sketch de Arduino lee correctamente el sensor DHT22, controla un LED indicador y notifica eventos de temperatura alta a un flujo externo en Make.com hacia Telegram. El siguiente paso natural es completar la documentación (video de funcionamiento, evidencias fotográficas del armado, gráficas y tablas con datos reales, y las conclusiones del experimento), además de resolver el manejo de credenciales antes de considerar el repositorio como una versión final o de entrega.
